

Technical Report: Electrical Line Analysis and Design

Project for Medium and Low Voltage Electrical Installations

Student Name

October 5, 2025
University of Almería

Contents

Chapter 1

Conductor Heating Analysis

1.1 Introduction

The analysis of conductor heating is fundamental to determining the maximum current-carrying capacity (ampacity) of an electrical cable. When current flows through a conductor, energy is dissipated as heat due to the material's resistance (Joule heating). This generated heat must be effectively dissipated to the surrounding environment. An equilibrium is reached when the rate of heat generation equals the rate of heat dissipation, resulting in a stable operating temperature.

This analysis ensures that the conductor's temperature does not exceed the maximum permissible limit for its insulating material (e.g., 70 °C for PVC, 90 °C for XLPE), preventing premature aging, degradation, and potential failure. This section details the methodology used to model both steady-state and transient thermal behavior for conductors in various installation scenarios.

1.2 Methodology

The thermal model is based on the principle of heat balance.

1.2.1 Steady-State Analysis

In steady-state, the heat generated within the conductor is equal to the heat dissipated from its surface.

$$P_{\text{generated}} = P_{\text{dissipated}} \quad (1.1)$$

Heat generation is calculated using the Joule effect, where resistance $R(T)$ is a function of temperature:

$$P_{\text{generated}} = I^2 \cdot R(T) = I^2 \cdot R_{20}[1 + \alpha(T - T_{20})] \quad (1.2)$$

Heat dissipation depends on the installation environment. For a cylindrical cable, the heat conducted through the insulation is given by:

$$P_{\text{dissipated}} = \frac{2\pi kL(T_c - T_s)}{\ln(r_o/r_i)} \quad (1.3)$$

where T_c is the conductor temperature and T_s is the surface temperature. The heat dissipated from the surface to the environment varies by scenario (convection, radiation, conduction to soil).

1.2.2 Transient Analysis

During heat-up, the difference between generated and dissipated power contributes to the increase in the conductor's internal energy. This is described by the differential equation:

$$mc_p \frac{dT}{dt} = P_{\text{generated}}(T) - P_{\text{dissipated}}(T) \quad (1.4)$$

This equation is solved numerically to obtain the temperature profile over time.

1.3 Results

The developed MATLAB suite was used to analyze an overhead aluminum conductor with XLPE insulation.

Table 1.1: Sample Results for Overhead ACSR Conductor Analysis.

Parameter	Value
Conductor Material	ACSR
Insulation Type	XLPE (90 °C max)
Cross-Section	50 mm ²
Ambient Temperature	40 °C
Resistance at 20 °C (AC)	0.0812 Ω
Max Current (Ampacity)	207.3 A

The steady-state and transient thermal behaviors are visualized in Figure ?? and ??.

Figure 1.1: Steady-state temperature as a function of current. (Placeholder for MATLAB plot)

Figure 1.2: Transient temperature rise of conductor and insulator. (Placeholder for MATLAB plot)

Chapter 2

Voltage Drop Analysis

2.1 Introduction

Voltage drop in an electrical installation is the reduction in voltage from the source to the point of consumption. It is caused by the impedance of the electrical line. Excessive voltage drop can lead to poor performance or damage to electrical equipment, reduced efficiency, and lighting flicker.

Regulatory standards, such as the Spanish Low Voltage Electrotechnical Regulation (REBT), define strict limits on the maximum allowable voltage drop to ensure safety and quality of supply. This analysis implements the Blondel formula to accurately calculate voltage drop and assesses compliance against these standards for both single-line and complete multi-segment installations.

2.2 Methodology

The core of the analysis is the Blondel formula, which accounts for both resistance and reactance.

2.2.1 Blondel Formula

The voltage drop (ΔU) is calculated as:

$$\Delta U = K \cdot L \cdot I \cdot (R \cos \phi + X \sin \phi) \quad (2.1)$$

where K is a phase factor ($\sqrt{3}$ for three-phase, 2 for single-phase), L is length, I is current, R is resistance per meter, X is reactance per meter, and $\cos \phi$ is the power factor.

2.2.2 AC Resistance

The resistance R is adjusted for AC circuits to account for skin and proximity effects, which increase the effective resistance compared to its DC value.

$$R_{AC} = R_{DC}(1 + k_{\text{skin}} + k_{\text{proximity}}) \quad (2.2)$$

These factors are calculated based on IEC 60287.

2.2.3 Installation Analysis

The analysis can be performed in two modes:

- 1. **Single Line:** A simple analysis of a single circuit from source to load.
- 2. **Full Installation:** A cumulative analysis across multiple standard segments of an installation, from the LV distribution line to the internal circuits.

2.3 Results

A full installation analysis was conducted for a three-phase system with a final load of 50 A. The results are summarized in Table ?? and visualized in Figure ??.

Table 2.1: Voltage Drop Summary for a Complete Installation.

Segment	Length (m)	Drop (V)	Voltage In (V)	Voltage Out (V)
LV Distribution Line	50.0	1.85	400.00	398.15
Individual Connection Line	20.0	1.10	398.15	397.05
Main Feeder Line (LGA)	15.0	1.25	397.05	395.80
Individual Branch Line (DI)	25.0	3.20	395.80	392.60
Internal Circuits	15.0	3.55	392.60	389.05
Total Cumulative Drop	125.0	10.95	400.00	389.05

The cumulative drop is 10.95 V, which represents a **2.74%** drop from the source voltage, well within the REBT limits.

Figure 2.1: Step-wise voltage drop across installation segments. (Placeholder for MATLAB plot)

Chapter 3

Conductor Cross-Section Sizing

3.1 Introduction

Proper conductor sizing is a critical design step that balances safety, performance, and cost. An undersized conductor can overheat or cause excessive voltage drop, while an oversized conductor is uneconomical. The optimal cross-section is the smallest standard size that satisfies all technical criteria, or the size that provides the lowest total lifecycle cost.

This analysis module implements three criteria for sizing:

1. **Voltage Drop Criterion:** For long lines where voltage drop is the limiting factor.
2. **Current-Carrying Capacity Criterion:** For short, heavily loaded lines where heating is the primary concern.
3. **Economic Optimization:** A combined analysis that minimizes the total cost of ownership over the cable's lifespan.

3.2 Methodology

3.2.1 Technical Sizing

The minimum section required by voltage drop (s_{vd}) is calculated by rearranging the Blondel formula. The minimum section required by ampacity (s_{amp}) is found by selecting the smallest standard conductor with a current rating greater than the load current. The final technical section is the larger of the two.

$$s_{\text{technical}} = \max(s_{vd}, s_{amp}) \quad (3.1)$$

3.2.2 Economic Optimization

The lifecycle cost (LCC) is the sum of the initial purchase and installation cost and the cumulative cost of energy lost to heat over the conductor's lifetime.

$$\text{LCC} = C_{\text{cable}} + C_{\text{losses}} \quad (3.2)$$

where the cost of losses is calculated as:

$$C_{\text{losses}} = N \cdot I^2 \cdot R \cdot (\text{hours/year}) \cdot (\text{years}) \cdot (\text{cost/kWh}) \quad (3.3)$$

An economic analysis is performed on all technically compliant conductor sizes to find the section with the minimum LCC.

3.3 Results

An economic analysis was performed for a three-phase copper line. The technically required minimum section was determined to be 16 mm^2 . The economic analysis, however, shows that a larger conductor is more cost-effective over a 20-year lifespan.

Figure 3.1: Lifecycle cost comparison. The economically optimal section is 25 mm^2 , which is larger than the technical minimum of 16 mm^2 . (Placeholder for MATLAB plot)

Appendix A

References

1. **IEC 60287-1-1:** *Electric cables - Calculation of the current rating - Part 1-1: Current rating equations (100 % load factor) and calculation of losses - General.* International Electrotechnical Commission.
2. **IEEE 738-2012:** *IEEE Standard for Calculating the Current-Temperature Relationship of Bare Overhead Conductors.* Institute of Electrical and Electronics Engineers.
3. **Reglamento Electrotécnico para Baja Tensión (REBT):** *ITC-BT-19 Instalaciones interiores o receptoras. Prescripciones generales.* Ministerio de Industria, Turismo y Comercio, Spain.

Appendix B

Material and Assumed Properties

Table B.1: Properties of Conductor and Insulating Materials.

Material	Property	Symbol	Value & Unit
Copper	Electrical Conductivity	σ	56 S m/mm ²
	Density	ρ_{den}	8960 kg/m ³
	Specific Heat	c_p	385 J/(kg · K)
Aluminum	Electrical Conductivity	σ	36 S m/mm ²
	Density	ρ_{den}	2700 kg/m ³
	Specific Heat	c_p	900 J/(kg · K)
PVC	Max Temperature	T_{max}	70 °C
	Thermal Conductivity	k	0.19 W/(m · K)
	Specific Heat	c_p	1000 J/(kg · K)
XLPE	Max Temperature	T_{max}	90 °C
	Thermal Conductivity	k	0.35 W/(m · K)
	Specific Heat	c_p	2300 J/(kg · K)
Soil	Thermal Resistivity	ρ_{soil}	1.2 K · m/W (Assumed)

Table B.2: Assumed Values for Economic Analysis.

Parameter	Value	Unit
Lifecycle Analysis Period	20	years
Annual Operating Hours	4000	hours/year
Cost of Electricity	0.15	€/kWh

Appendix C

List of Formulas

1. **Temperature-Corrected Resistance:** $R_T = R_{ref}[1 + \alpha(T - T_{ref})]$
2. **Heat Generation (Joule Heating):** $P_{gen} = I^2 R_T$
3. **Cylindrical Heat Conduction (Insulation):** $P_{diss} = \frac{2\pi k L (T_{conductor} - T_{surface})}{\ln(r_{outer}/r_{inner})}$
4. **Transient Heat Balance:** $mc_p \frac{dT}{dt} = P_{gen} - P_{diss}$
5. **Voltage Drop (Blondel):** $\Delta U = K \cdot L \cdot I \cdot (R \cos \phi + X \sin \phi)$
6. **AC Resistance:** $R_{AC} = R_{DC}(1 + k_{skin} + k_{proximity})$
7. **Sizing by Voltage Drop (3-Phase):** $s = \frac{\sqrt{3} \cdot L \cdot I \cdot \cos \phi}{\sigma \cdot \Delta U_{max}}$
8. **Lifecycle Cost:** $LCC = C_{cable} + C_{losses}$

MATLAB Code

```

1 %
2 % =====
3 % MASTER SCRIPT for Electrical Line Design Suite (V4 - Fully Integrated)
4 %
5 % MODIFIED:
6 % - Added Case 3 for 'Conductor Sizing Analysis'.
7 % - The menu for Case 3 now reflects the new student criteria.
8 % - Calls the new C1_ConductorSizingAnalysis.m script.
9 %
10 % =====
11 %% --- Cleanup and Initialization ---
12 clc;
13 clear;
14 close all;
15 %% --- User Selection of Analysis Case ---
16 analysisChoice = centeredMenu('Select which project case to work with:',
17     ...
18     'Conductor Heating Analysis', ...
19     'Voltage Drop Analysis', ...
20     'Conductor Sizing Analysis');
21 %% --- Run the Selected Analysis Script ---
22 switch analysisChoice
23     case 1 % --- CONDUCTOR HEATING ---
24         scenarioChoice = centeredMenu('Select Heating Scenario:', '
Underground (Student A)', 'Overhead (Student B)', 'Building/Grouped (
Student C)');
25         if scenarioChoice > 0
26             assignin('base', 'installationChoice', scenarioChoice);
27             disp('--- Launching Conductor Heating Analysis ---');
28             run('A1_ConductorHeatingAnalysis.m');
29             evalin('base', 'clear installationChoice');
30         end
31

```

```

32     case 2 % --- VOLTAGE DROP ---
33         analysisMode = centeredMenu('Select Voltage Drop Analysis Type
: ', 'Full Installation Analysis (Multi-Segment)', 'Simple Single Line
Analysis');
34         if analysisMode > 0
35             assignin('base', 'vd_analysisMode', analysisMode);
36             disp('--- Launching Voltage Drop Analysis ---');
37             run('B1_VoltageDropAnalysis.m');
38             evalin('base', 'clear vd_analysisMode');
39         end
40
41     case 3 % --- CONDUCTOR SIZING ---
42         scenarioChoice = centeredMenu('Select Sizing Criterion:', '
Voltage Drop (Student A)', 'Current-Carrying Capacity (Student B)', '
Economic Optimization (Student C)');
43         if scenarioChoice > 0
44             assignin('base', 'sizing_scenarioChoice', scenarioChoice);
45             disp('--- Launching Conductor Sizing Analysis ---');
46             run('C1_ConductorSizingAnalysis.m');
47             evalin('base', 'clear sizing_scenarioChoice');
48         end
49
50     case 0
51         disp('No analysis selected. Exiting.');
```

```

52 end
```

D.2 A1_ConductorHeatingAnalysis.m

```

1 %
=====

2 % MAIN SCRIPT for Conductor Thermal Analysis (V5.1 - Insulator Transient
)
3 %
=====

4 % MODIFIED:
5 % - Added a post-processing step to calculate the transient temperature
6 %   of the insulator based on the conductor's temperature.
7 % - Updated the final plot to display both conductor and insulator
8 %   temperature curves for comparison.
9 % - Added density constants for insulating materials.
10 %
=====

11
12 %% --- Cleanup and Initialization ---
13 % clc; clear; close all; % Commented out for master script control
14
15 %% --- User Input: Scenario Selection ---
16 if ~exist('installationChoice', 'var')
17     installationChoice = centeredMenu('Select Installation Scenario:', '
Underground (Student A)', 'Overhead (Student B)', 'Building/Grouped (
Student C)');
```

```

18 end
19
```

```

20 %% --- Define Physical Constants ---
21 T_ref = 20; % Reference temperature for resistance [C]
22 % Material Properties (Density [kg/m^3], Specific Heat [J/(kg*C)])
23 rho_den_copper = 8960; cp_copper = 385;
24 rho_den_aluminum = 2700; cp_aluminum = 900;
25 rho_den_pvc = 1400; cp_pvc = 1000;
26 rho_den_xlpe = 920; cp_xlpe = 2300;
27
28 %% --- Scenario-Specific User Input ---
29 envParams = struct();
30 switch installationChoice
31     case 1 % Underground
32         envParams.scenario = 'underground';
33         disp('--- UNDERGROUND INSTALLATION INPUT ---');
34         envParams.T_env = input('Enter ground temperature [C] (e.g., 15)
: ');
35         envParams.rho_soil = input('Enter soil thermal resistivity [K*m/
W] (e.g., 1.2): ');
36         envParams.burial_depth = input('Enter burial depth to cable
center [m] (e.g., 1): ');
37     case 2 % Overhead
38         envParams.scenario = 'overhead';
39         disp('--- OVERHEAD INSTALLATION INPUT ---');
40         envParams.T_env = input('Enter ambient air temperature [C] (e.g
., 40): ');
41         envParams.wind_speed = input('Enter wind speed [m/s] (e.g., 0.5)
: ');
42         envParams.solar_irradiance = input('Enter solar irradiance [W/m
^2] (e.g., 1000): ');
43         envParams.emissivity = 0.9;
44         envParams.absorptivity = 1.0;
45     case 3 % Building/Grouped
46         envParams.scenario = 'building';
47         disp('--- BUILDING/GROUPED INSTALLATION INPUT ---');
48         envParams.T_env = input('Enter ambient air temperature [C] (e.g
., 30): ');
49         num_cables = input('Enter total number of cables in the group (e
.g., 6): ');
50
51         if num_cables <= 1, k_g = 1.0;
52         elseif num_cables <= 3, k_g = 0.80;
53         elseif num_cables <= 6, k_g = 0.70;
54         elseif num_cables <= 9, k_g = 0.60;
55         elseif num_cables <= 20, k_g = 0.50;
56         else, k_g = 0.45;
57         end
58         envParams.grouping_factor = k_g;
59
60         envParams.wind_speed = 0;
61         envParams.solar_irradiance = 0;
62         envParams.emissivity = 0.9;
63         envParams.absorptivity = 1.0;
64         fprintf('Applied grouping derating factor: %.2f\n', envParams.
grouping_factor);
65     case 0
66         disp('No scenario selected. Exiting script.');
```

```

69 end
70
71 %% --- General User Input ---
72 disp('--- GENERAL CONDUCTOR INPUT ---');
73 lineLength = input('Enter line length [m] (e.g., 100): ');
74 crossSection = input('Enter cross-sectional area [mm^2] (e.g., 50): ');
75 insulatorThickness = input('Enter insulator thickness [mm] (e.g., 1.2): ');
76 frequency = input('Enter AC frequency [Hz] (e.g., 50): ');
77 materialChoice = centeredMenu('Select Conductor Material:', 'Copper', 'Aluminum', 'ACSR');
78 insulationChoice = centeredMenu('Select Insulation Type:', 'PVC', 'XLPE');
79
80 if materialChoice == 0 || insulationChoice == 0
81     disp('No material or insulation selected. Exiting script.');
```

return;

```

82 end
83
84 %% --- Set Conductor & Insulation Parameters ---
85 alpha = 0.00393;
86 switch materialChoice, case 1, sigma = 56; materialName = 'Copper';
    density_cond = rho_den_copper; cp_cond = cp_copper; case 2, sigma =
    36; materialName = 'Aluminum'; density_cond = rho_den_aluminum;
    cp_cond = cp_aluminum; case 3, sigma = 36; materialName = 'ACSR';
    density_cond = rho_den_aluminum; cp_cond = cp_aluminum; end
87 switch insulationChoice, case 1, T_mat = 70; k = 0.19; insulationName =
    'PVC'; density_ins = rho_den_pvc; cp_ins = cp_pvc; case 2, T_mat =
    90; k = 0.35; insulationName = 'XLPE'; density_ins = rho_den_xlpe;
    cp_ins = cp_xlpe; end
88 envParams.k_insulator = k;
89
90 %% --- Resistance and Geometry Calculations ---
91 r_inner_m = sqrt(crossSection / pi) / 1000;
92 r_outer_m = r_inner_m + (insulatorThickness / 1000);
93 R_20_DC = lineLength / (sigma * crossSection);
94 [k_skin, k_proximity] = A3b_AC_Resistance_Factor(frequency, r_inner_m *
    2, sigma);
95 R_20_AC = R_20_DC * (1 + k_skin + k_proximity);
96
97 %% --- Analysis & Display ---
98 fprintf('\n--- ANALYSIS FOR SCENARIO: %s ---\n', upper(envParams.
    scenario));
99 I_max_DC = A2_MaximumCurrent(R_20_DC, alpha, T_mat, envParams,
    lineLength, r_inner_m, r_outer_m);
100 I_max_AC = A2_MaximumCurrent(R_20_AC, alpha, T_mat, envParams,
    lineLength, r_inner_m, r_outer_m);
101 fprintf('Max Current (Ampacity) - DC Resistance: %.2f A\n', I_max_DC);
102 fprintf('Max Current (Ampacity) - AC Resistance: %.2f A\n', I_max_AC);
103 if isfield(envParams, 'grouping_factor')
104     fprintf('NOTE: Above values INCLUDE the grouping derating factor of
        %.2f.\n', envParams.grouping_factor);
105 end
106 fprintf('-----\n');
107
108 A5_HeatingCurves(I_max_AC, T_mat, envParams, R_20_DC, R_20_AC, alpha,
    T_ref, lineLength, r_inner_m, r_outer_m, materialName, insulationName
    );
109

```

```

110 %% --- Transient Analysis (Conductor and Insulator) ---
111 sim_current_prompt = sprintf('Enter a current for transient simulation (
    e.g., %.0f A): ', I_max_AC * 0.8);
112 sim_current = input(sim_current_prompt);
113 if isempty(sim_current), sim_current = I_max_AC * 0.8; end
114
115 % Conductor mass calculation
116 volume_cond = (crossSection / 1e6) * lineLength;
117 mass_cond = volume_cond * density_cond;
118 time_span = [0 7200]; % Simulate for 2 hours
119
120 % Run primary simulation for conductor temperature
121 [time_ac, temp_cond_ac] = A6_TransientHeating(sim_current, time_span,
    envParams, R_20_AC, alpha, T_ref, lineLength, r_inner_m, r_outer_m,
    mass_cond, cp_cond);
122
123 % --- NEW: Post-processing to find insulator temperature ---
124 fprintf('Calculating insulator transient temperature...\n');
125 temp_ins_avg_ac = zeros(size(time_ac));
126 options = optimset('Display','off');
127 % Thermal resistance of the insulation layer
128 R_thermal_ins = log(r_outer_m / r_inner_m) / (2 * pi * envParams.
    k_insulator * lineLength);
129
130 for i = 1:length(time_ac)
131     T_c = temp_cond_ac(i);
132     % At any instant, heat flow through insulation equals heat
    dissipated from surface
133     % (Tc - Ts)/R_ins = A7_HeatDissipation(Ts, ...)
134     % We need to find the surface temp (Ts) that balances this equation
135     balance_eq = @(T_s) (T_c - T_s) / R_thermal_ins - A7_HeatDissipation
    (T_s, envParams, r_outer_m, lineLength);
136     try
137         T_s = fzero(balance_eq, T_c, options); % Find the surface
        temperature
138         % Approximate average insulator temp as the mean of conductor
        and surface temp
139         temp_ins_avg_ac(i) = (T_c + T_s) / 2;
140     catch
141         temp_ins_avg_ac(i) = T_c; % If solver fails, assume same temp
142     end
143 end
144 fprintf('Calculation complete.\n');
145
146 % --- UPDATED: Plotting both conductor and insulator temperatures ---
147 figure;
148 hold on;
149 plot(time_ac/60, temp_cond_ac, 'r-', 'LineWidth', 2, 'DisplayName', '
    Conductor Temp');
150 plot(time_ac/60, temp_ins_avg_ac, 'm-', 'LineWidth', 2, 'DisplayName', '
    Insulator Avg Temp');
151
152 T_ss_ac = A4_ThermalEquilibrium(sim_current, R_20_AC, alpha, T_ref,
    envParams, lineLength, r_inner_m, r_outer_m);
153 line([0, time_ac(end)/60], [T_ss_ac, T_ss_ac], 'Color', 'k', 'LineStyle
    ', '--', 'DisplayName', sprintf('Final Equilibrium (%.1f C)', T_ss_ac
    ));
154

```



```

155 title(sprintf('Transient Heating Comparison for %.1f A', sim_current));
156 xlabel('Time (minutes)');
157 ylabel('Temperature (C)');
158 grid on;
159 legend('show', 'Location', 'southeast');
160 hold off;

```

D.3 A2_MaximumCurrent.m

```

1 function I_max = A2_MaximumCurrent(R_20, alpha, T_mat, envParams,
   lineLength, r_inner, r_outer)
2 %
   =====
3 % FUNCTION: A2_MaximumCurrent (V4)
4 %
   =====
5 % Description:
6 % Calculates the maximum allowable continuous current (Ampacity). This
   is
7 % determined by finding the current that causes the conductor to reach
   its
8 % maximum rated temperature (T_mat) in a given environment.
9 %
10 % MODIFIED:
11 % - It no longer calculates dissipation directly. Instead, it calls the
12 %   central A7_HeatDissipation module.
13 % - This makes the function scenario-independent.
14 %
   =====
15
16 % 1. Calculate the conductor's resistance at its maximum temperature
17 R_Tmat = A3_ResistanceTemperatureCorrection(R_20, alpha, T_mat, T_ref);
18
19 % 2. Calculate the total heat dissipation from the cable surface when
   the
20 %   conductor is at T_mat. This requires finding the surface
   temperature.
21 options = optimset('Display','off');
22 R_thermal_ins = log(r_outer / r_inner) / (2 * pi * envParams.k_insulator
   * lineLength);
23 balance_eq = @(T_s) (T_mat - T_s) / R_thermal_ins - A7_HeatDissipation(
   T_s, envParams, r_outer, lineLength);
24 T_surface_at_max = fzero(balance_eq, T_mat, options);
25
26 P_dissipated_max = A7_HeatDissipation(T_surface_at_max, envParams,
   r_outer, lineLength);
27
28 % 3. Solve for I_max from the heat balance equation: P_gen = P_diss
29 %   I_max^2 * R_Tmat = P_dissipated_max
30 if R_Tmat <= 0
31     I_max = inf; % Avoid division by zero if resistance is non-positive
32 else
33     I_max = sqrt(P_dissipated_max / R_Tmat);

```

```

34 end
35
36 end

```

D.4 A3_ResistanceTemperatureCorrection.m

```

1 function R_T = A3_ResistanceTemperatureCorrection(R_20, alpha, T, T_ref)
2 %
3 % =====
4 % FUNCTION: correctResistanceForTemp
5 %
6 % Description:
7 % Adjusts the electrical resistance of a conductor based on its
8 % operating
9 % temperature using the linear approximation formula.
10 %
11 % Formula:
12 %  $R_T = R_{20} * (1 + \alpha * (T - T_{ref}))$ 
13 %
14 % Inputs:
15 % R_20 - Resistance at the reference temperature (T_ref) [Ohm]
16 % alpha - Temperature coefficient of resistance [1/C]
17 % T - The new operating temperature [C]
18 % T_ref - The reference temperature (typically 20C) [C]
19 %
20 % Output:
21 % R_T - The corrected resistance at temperature T [Ohm]
22 %
23 % =====
24
25 R_T = R_20 * (1 + alpha * (T - T_ref));
26
27 end

```

D.5 A3b_AC_Resistance_Factor.m

```

1 function [k_skin, k_proximity] = A3b_AC_Resistance_Factor(freq, diameter
2 , sigma)
3 %
4 % =====
5 % FUNCTION: calculateACResistanceFactor (V2 - Corrected)
6 %
7 % Description:
8 % Calculates the skin effect (ys or k_skin) and proximity effect (yp or
9 % k_proximity) factors based on the IEC 60287 standard. These factors
10 % are
11 % used to determine AC resistance from DC resistance.

```

```

9 % R_AC = R_DC * (1 + k_skin + k_proximity)
10 %
11 % Inputs:
12 %   freq      - System frequency [Hz]
13 %   diameter  - Conductor outer diameter [m]
14 %   sigma     - Electrical conductivity of the material [S*m/mm^2]
15 %
16 % Outputs:
17 %   k_skin     - Skin effect factor (dimensionless)
18 %   k_proximity - Proximity effect factor (dimensionless)
19 %
20 % =====
21 % --- Skin Effect Calculation (Based on IEC 60287-1-1) ---
22 %
23 % 1. Calculate DC resistance per meter [Ohm/m]
24 cross_section_m2 = pi * (diameter/2)^2; % [m^2]
25 sigma_Sm = sigma * 1e6; % Convert [S*m/mm^2] to [S/m]
26 R_dc_per_meter = 1 / (sigma_Sm * cross_section_m2);
27
28 % 2. Calculate the skin effect parameter x_s^2
29 if R_dc_per_meter == 0
30     x_s_sq = 0;
31 else
32     x_s_sq = (8 * pi * freq / R_dc_per_meter) * 1e-7;
33 end
34
35 % 3. Calculate the skin effect factor k_skin (also called ys)
36 k_skin = (x_s_sq^2) / (192 + 0.8 * x_s_sq^2);
37
38 % --- Proximity Effect Calculation (Based on IEC 60287-1-1) ---
39 %
40 % 1. The proximity effect parameter x_p^2 is identical to x_s^2
41 x_p_sq = x_s_sq;
42
43 % 2. The proximity factor depends on cable arrangement. The following is
44 %    a
45 %    well-established approximation for three single-core cables in
46 %    trefoil
47 %    (touching) formation, where center-to-center spacing 's' equals '
48 %    diameter'.
49 %    For this arrangement, (diameter/spacing) = 1.
50 if x_p_sq == 0
51     k_proximity = 0;
52 else
53     k_proximity = (x_p_sq^2 / (192 + 0.8*x_p_sq^2)) * ...
54                   ( 0.312 + (1.18 / ((x_p_sq^2 / (192 + 0.8*x_p_sq^2)) +
55                     0.27)) );
56 end
57 end

```

D.6 A4_ThermalEquilibrium.m

```

1 function T_eq = A4_ThermalEquilibrium(I, R_20, alpha, T_ref, envParams,
   lineLength, r_inner, r_outer)
2 %
   =====
3 % FUNCTION: A4_ThermalEquilibrium (V4 - Iterative)
4 %
   =====
5 % Description:
6 % Calculates the equilibrium temperature (T_eq) of a conductor for a
   given
7 % current. It iteratively solves the heat balance equation:
8 %   P_generated(T_eq) = P_dissipated(T_eq)
9 %
10 % MODIFIED:
11 % - The solving method is now iterative using fzero, which is more
   robust
12 %   for complex dissipation models (like radiation and convection).
13 %
   =====
14
15 % Define the heat balance equation as a function handle.
16 % The root of this function (where it equals zero) is the equilibrium
   temp.
17 % balance(T) = P_generated(T) - P_dissipated(T)
18 heat_balance_eq = @(T) (A3_ResistanceTemperatureCorrection(R_20, alpha,
   T, T_ref) * I^2) ...
19     - A7_HeatDissipation(T, envParams, r_outer,
   lineLength);
20
21 % Use MATLAB's built-in root finder 'fzero' to solve for T_eq.
22 % We provide an initial guess, e.g., the ambient temperature.
23 try
24     options = optimset('Display', 'off'); % Suppress fzero output
25     T_eq = fzero(heat_balance_eq, envParams.T_env, options);
26 catch
27     % If fzero fails (e.g., no solution), return NaN
28     T_eq = NaN;
29     warning('Thermal equilibrium could not be found for I = %.1f A.', I)
   ;
30 end
31
32 end

```

D.7 A5_HeatingCurves.m

```

1 function A5_HeatingCurves(I_max, T_mat, envParams, R_20_DC, R_20_AC,
   alpha, T_ref, lineLength, r_inner, r_outer, materialName,
   insulationName)
2 %
   =====
3 % FUNCTION: A5_HeatingCurves (V5 - Corrected)

```

```

4 %
  =====

5 % Description:
6 % Generates plots showing the conductor's equilibrium temperature vs.
  current.
7 % MODIFIED: The input variable 'length' was renamed to 'lineLength' to
8 %           avoid conflict with MATLAB's built-in length() function,
  which
9 %           was causing a runtime error.
10 %
  =====

11
12 % 1. Create a vector of currents to plot
13 current_vector = linspace(0, I_max * 1.2, 100);
14
15 % 2. Calculate equilibrium temperatures for both DC and AC resistance
16 temp_vector_dc = zeros(size(current_vector));
17 temp_vector_ac = zeros(size(current_vector));
18
19 % The loop now correctly calls the built-in 'length' function
20 for i = 1:length(current_vector)
21     % The variable 'lineLength' is correctly passed to the next function
22     temp_vector_dc(i) = A4_ThermalEquilibrium(current_vector(i), R_20_DC
  , alpha, T_ref, envParams, lineLength, r_inner, r_outer);
23     temp_vector_ac(i) = A4_ThermalEquilibrium(current_vector(i), R_20_AC
  , alpha, T_ref, envParams, lineLength, r_inner, r_outer);
24 end
25
26 % 3. Create the plot
27 figure;
28 hold on;
29
30 % Plot the curves
31 plot(current_vector, temp_vector_dc, 'b-', 'LineWidth', 2, 'DisplayName
  ', 'Equilibrium Temp (DC)');
32 plot(current_vector, temp_vector_ac, 'r-', 'LineWidth', 2, 'DisplayName
  ', 'Equilibrium Temp (AC)');
33
34 % Plot safety and reference lines
35 line([0, I_max * 1.2], [T_mat, T_mat], 'Color', 'k', 'LineStyle', '--',
  'LineWidth', 1.5, 'DisplayName', sprintf('Max Temp (%.0fC)', T_mat));
36 line([I_max, I_max], [envParams.T_env, T_mat], 'Color', [0.4660 0.6740
  0.1880], 'LineStyle', ':', 'LineWidth', 1.5, 'DisplayName', sprintf('
  Max AC Current (%.1f A)', I_max));
37
38 % Add marker at the intersection
39 plot(I_max, T_mat, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 8, '
  HandleVisibility', 'off');
40
41 % Formatting
42 title(sprintf('Steady-State Heating Curve (%s)', envParams.scenario));
43 subtitle(sprintf('%s Conductor, %s Insulation', materialName,
  insulationName));
44 xlabel('Current (A)');
45 ylabel('Conductor Temperature (C)');
46 legend('show', 'Location', 'southeast');

```

```

47 grid on;
48 box on;
49 ylim([envParams.T_env - 10, T_mat + 20]);
50 xlim([0, I_max * 1.25]);
51 hold off;
52
53 end

```

D.8 A6_TransientHeating.m

```

1 function [time, temp] = A6_TransientHeating(I, time_span, envParams,
    R_20, alpha, T_ref, length, r_inner, r_outer, mass, cp)
2 %
    =====
3 % FUNCTION: A6_TransientHeating (V5 - FIXED)
4 %
    =====
5 % Description:
6 % Models the temperature of a conductor over time.
7 % FIXED: The call to the heat dissipation function has been corrected
    from
8 % the erroneous 'A7_HeatGain' to the correct 'A7_HeatDissipation'.
9 %
10 % ODE:  $m \cdot cp \cdot (dT/dt) = P_{\text{generated}}(T) - P_{\text{dissipated}}(T)$ 
11 %
    =====
12
13 % Define the ODE as a nested function for the solver
14 function dTdt = heat_balance_ode(~, T)
15     % 1. Heat Generation at current temperature T
16     R_T = A3_ResistanceTemperatureCorrection(R_20, alpha, T, T_ref);
17     P_generated = R_T * I^2;
18
19     % 2. Heat Dissipation at current temperature T
20     % FIXED: Corrected function call from A7_HeatGain to
    A7_HeatDissipation
21     P_dissipated = A7_HeatDissipation(T, envParams, r_outer, length)
    ;
22
23     % 3. Rate of change of internal energy (dT/dt)
24     dTdt = (P_generated - P_dissipated) / (mass * cp);
25 end
26
27 % Use MATLAB's ODE solver (ode45)
28 options = odeset('RelTol', 1e-6, 'AbsTol', 1e-6);
29 % The initial temperature for the simulation is the ambient temperature
30 [time, temp] = ode45(@heat_balance_ode, time_span, envParams.T_env,
    options);
31
32 end

```

D.9 A7_HeatDissipation.m

```
1 function P_diss = A7_HeatDissipation(T_conductor, envParams, r_outer,
2     length)
3
4     =====
5
6 % FUNCTION: A7_HeatDissipation (Module)
7 %
8 % =====
9
10 % Description:
11 % This is the central module for calculating the net heat dissipation
12 % from
13 % the conductor surface to the environment. It calls the appropriate
14 % physical model based on the installation scenario defined in envParams
15 %
16 % Inputs:
17 %   T_conductor - The temperature of the conductor surface [C]
18 %   envParams   - A struct containing all environment-specific
19 %                 parameters
20 %   r_outer     - The outer radius of the cable (including insulation) [
21 %                 m]
22 %   length      - The length of the conductor [m]
23 %
24 % Output:
25 %   P_diss      - The net heat dissipated from the cable [W].
26 %                 (Positive for net cooling, negative for net heating)
27 %
28 % =====
29
30 % The main function acts as a router to the correct sub-function
31 switch envParams.scenario
32     case 'underground'
33         P_diss = dissipation_underground(T_conductor, envParams.T_env,
34             envParams.rho_soil, envParams.burial_depth, r_outer, length);
35     case 'overhead'
36         P_diss = dissipation_overhead(T_conductor, envParams.T_env,
37             envParams.wind_speed, envParams.solar_irradiance, envParams.
38             emissivity, envParams.absorptivity, r_outer, length);
39     case 'building'
40         % For a building, we model it as overhead but with no wind/sun
41         % and apply a grouping factor
42         P_diss = dissipation_overhead(T_conductor, envParams.T_env, 0,
43             0, envParams.emissivity, envParams.absorptivity, r_outer, length);
44         % Apply the derating factor to the dissipation capability
45         if isfield(envParams, 'grouping_factor')
46             P_diss = P_diss * envParams.grouping_factor;
47         end
48 end
49 end
50
51 % --- SUB-FUNCTIONS for each scenario ---
52
53 function P_diss = dissipation_underground(T_conductor, T_soil, rho_soil,
54     H, r_outer, L)
```

```

40 % Model based on IEC 60287 for buried cables (simplified)
41 %  $P = (T_{\text{conductor}} - T_{\text{ambient}}) / R_{\text{thermal}}$ 
42 %  $R_{\text{thermal}}$  for a single buried cable is  $\rho_{\text{soil}} / (2\pi) * \log(2H/r)$ 
43     if  $H \leq r_{\text{outer}}$ 
44          $P_{\text{diss}} = 0$ ; % Cable is not buried, avoid log error
45         return;
46     end
47     thermal_resistance = ( $\rho_{\text{soil}} / (2 * \pi)$ ) *  $\log(2 * H / r_{\text{outer}})$ ;
48      $P_{\text{diss}} = (L * (T_{\text{conductor}} - T_{\text{soil}})) / \text{thermal\_resistance}$ ;
49 end
50
51 function  $P_{\text{diss}} = \text{dissipation\_overhead}(T_{\text{conductor}}, T_{\text{air}}, V_{\text{wind}},$ 
     $Q_{\text{solar}}, \epsilon, \alpha, r_{\text{outer}}, L)$ 
52 % Model based on IEEE 738 standard for overhead lines
53      $D_{\text{outer}} = r_{\text{outer}} * 2$ ;
54      $P_{\text{conv}} = \text{convection\_overhead}(T_{\text{conductor}}, T_{\text{air}}, V_{\text{wind}}, D_{\text{outer}}, L)$ 
55     ;
56      $P_{\text{rad}} = \text{radiation\_overhead}(T_{\text{conductor}}, T_{\text{air}}, \epsilon, D_{\text{outer}}, L)$ ;
57      $P_{\text{solar}} = \text{solar\_gain}(Q_{\text{solar}}, \alpha, D_{\text{outer}}, L)$ ;
58     % Net dissipation is cooling (convection + radiation) minus heating
    (solar)
59      $P_{\text{diss}} = (P_{\text{conv}} + P_{\text{rad}}) - P_{\text{solar}}$ ;
60 end
61
62 function  $P_{\text{conv}} = \text{convection\_overhead}(T_{\text{conductor}}, T_{\text{air}}, V_{\text{wind}},$ 
     $D_{\text{outer}}, L)$ 
63 % Simplified convection model
64      $k_{\text{air}} = 0.026$ ; % Thermal conductivity of air [W/(m*K)]
65
66     % Nusselt number correlations (simplified)
67     if  $V_{\text{wind}} > 0$ 
68         % Forced convection
69          $Re = V_{\text{wind}} * D_{\text{outer}} / 1.5e-5$ ; % Reynolds number, kinematic
        viscosity of air  $\sim 1.5e-5$ 
70          $Nu = 0.3 + (0.62 * Re^{0.5} * 0.71^{0.33}) / (1 + (0.4/0.71)^{0.66})$ 
         $^{0.25}$ ; % Prandtl for air  $\sim 0.71$ 
71     else
72         % Natural convection
73          $Gr = 9.81 * (1/(T_{\text{air}}+273.15)) * \text{abs}(T_{\text{conductor}} - T_{\text{air}}) *$ 
         $D_{\text{outer}}^3 / (1.5e-5)^2$ ; % Grashof
74          $Nu = (0.6 + (0.387 * (Gr*0.71)^{0.166}) / (1 + (0.559/0.71)^{0.562})$ 
         $^{0.296})^2$ ;
75     end
76
77      $h = (k_{\text{air}} / D_{\text{outer}}) * Nu$ ; % Convective heat transfer coefficient
78     surface_area =  $\pi * D_{\text{outer}} * L$ ;
79      $P_{\text{conv}} = h * \text{surface\_area} * (T_{\text{conductor}} - T_{\text{air}})$ ;
80 end
81
82 function  $P_{\text{rad}} = \text{radiation\_overhead}(T_{\text{conductor}}, T_{\text{air}}, \epsilon, D_{\text{outer}}$ 
    ,  $L)$ 
83 % Radiation based on Stefan-Boltzmann law
84      $\sigma_{\text{sb}} = 5.67e-8$ ; % Stefan-Boltzmann constant [W/(m2*K4)]
85     surface_area =  $\pi * D_{\text{outer}} * L$ ;
86
87     % Temperatures must be in Kelvin
88      $T_{\text{cond\_K}} = T_{\text{conductor}} + 273.15$ ;

```



```

89     T_air_K = T_air + 273.15;
90
91     P_rad = sigma_sb * epsilon * surface_area * (T_cond_K^4 - T_air_K^4)
92     ;
93 end
94
95 function P_solar = solar_gain(Q_solar, alpha, D_outer, L)
96 % Solar heat gain on the projected area of the conductor
97     projected_area = D_outer * L;
98     P_solar = Q_solar * alpha * projected_area;
99 end

```

D.10 B1_VoltageDropAnalysis.m

```

1 %
2 =====
3 % MAIN SCRIPT for Voltage Drop Analysis (V3.1 - Mode Selection)
4 %
5 % =====
6
7 % MODIFIED:
8 % - Added a selection menu at the start to allow the user to choose
9 %   between a 'Full Installation' analysis (multi-segment) and a
10 %   'Single Line' analysis (the previous, simpler version).
11 % - The script now contains both workflows.
12 %
13 % =====
14
15 %% --- Cleanup ---
16 % clc; clear; close all; % Controlled by master script
17
18 %% --- User Selection of Analysis Mode ---
19 if ~exist('vd_analysisMode', 'var')
20     vd_analysisMode = centeredMenu('Select Voltage Drop Analysis Type:',
21     'Full Installation Analysis (Multi-Segment)', 'Simple Single Line
22     Analysis');
23 end
24
25 if vd_analysisMode == 0
26     disp('No analysis type selected. Exiting.');
```

```

34 %% =====
35 %                               WORKFLOW 1: FULL INSTALLATION
36 % =====

37 function run_full_installation_analysis()
38     % --- Initialize Installation Structure ---
39     installationSegments = {
40         'LV Distribution Line',
41         'Individual Connection Line',
42         'Main Feeder Line (LGA)',
43         'Individual Branch Line (DI)',
44         'Internal Circuits'
45     };
46     numSegments = length(installationSegments);
47     results = cell(numSegments, 1);
48
49     % --- Define Physical Constants ---
50     rho_copper = 0.0172; sigma_copper = 56;
51     rho_aluminum = 0.0282; sigma_aluminum = 36;
52
53     % --- Get Global Parameters ---
54     disp('--- Entering Global Parameters for Full Installation ---');
55     U_source_initial = input('Enter source voltage at LV Distribution
Line [V] (e.g., 230 or 400): ');
56     loadCurrent = input('Enter final load current [A] (e.g., 30): ');
57     cos_phi = input('Enter final load power factor (e.g., 0.95): ');
58     frequency = input('Enter AC frequency [Hz] (e.g., 50): ');
59
60     lineTypeChoice = centeredMenu('Select Dominant System Type:', '
Single-Phase', 'Three-Phase');
61     if lineTypeChoice == 1, lineType = 'single-phase'; else, lineType =
'three-phase'; end
62
63     % --- Data Collection Loop for Each Segment ---
64     voltage_at_start_of_segment = U_source_initial;
65     for i = 1:numSegments
66         fprintf('\n--- Input for Segment %d: %s ---\n', i,
installationSegments{i});
67         segmentData.name = installationSegments{i};
68         segmentData.length = input(sprintf('Enter length [m] for %s: ',
segmentData.name));
69         segmentData.crossSection = input(sprintf('Enter cross-section [
mm^2] for %s: ', segmentData.name));
70         materialChoice = centeredMenu(['Select Material for '
segmentData.name], 'Copper', 'Aluminum');
71         if materialChoice == 0, disp('Selection cancelled. Aborting.');
```

```

return; end
72         switch materialChoice, case 1, segmentData.resistivity =
rho_copper; segmentData.sigma = sigma_copper; case 2, segmentData.
resistivity = rho_aluminum; segmentData.sigma = sigma_aluminum; end
73         segmentData.reactance_per_meter = input(sprintf('Enter reactance
[Ohm/m] for %s (e.g., 0.0001): ', segmentData.name));
74
75         r_dc = segmentData.resistivity / segmentData.crossSection;
76         diameter_m = sqrt(4 * segmentData.crossSection / pi) / 1000;

```

```

77     [k_skin, k_prox] = A3b_AC_Resistance_Factor(frequency,
78     diameter_m, segmentData.sigma);
79     r_ac = r_dc * (1 + k_skin + k_prox);
80     segmentData.voltageDrop = B2_calculateExactVoltageDrop(lineType,
81     segmentData.length, loadCurrent, r_ac, segmentData.
82     reactance_per_meter, cos_phi);
81     segmentData.voltage_in = voltage_at_start_of_segment;
82     segmentData.voltage_out = voltage_at_start_of_segment -
83     segmentData.voltageDrop;
84     voltage_at_start_of_segment = segmentData.voltage_out;
85     results{i} = segmentData;
86     end
87     % --- Display Summary and Compliance ---
88     display_summary_table(results, U_source_initial);
89     B4b_checkInstallationCompliance(results, U_source_initial);
90     B6_plotInstallationProfile(results, U_source_initial);
91 end
92
93 function display_summary_table(results, U_source_initial)
94     fprintf('\n\n--- COMPLETE INSTALLATION VOLTAGE DROP SUMMARY ---\n');
95     fprintf
96     ('=====
97     n');
98     fprintf('%-30s | %-10s | %-10s | %-12s | %-12s\n', 'Segment', '
99     Length (m)', 'Drop (V)', 'Voltage In (V)', 'Voltage Out (V)');
100     fprintf
101     ('-----
102     n');
103     cumulativeDrop = 0;
104     for i = 1:length(results)
105         data = results{i};
106         cumulativeDrop = cumulativeDrop + data.voltageDrop;
107         fprintf('%-30s | %-10.1f | %-10.2f | %-12.2f | %-12.2f\n', data.
108         name, data.length, data.voltageDrop, data.voltage_in, data.
109         voltage_out);
110     end
111     fprintf
112     ('=====
113     n');
114     fprintf('Total Voltage Drop from Source to Final Load: %.2f V (%.2f
115     %%) \n', cumulativeDrop, (cumulativeDrop/U_source_initial)*100);
116 end
117
118 %%
119 =====
120
121 %                               WORKFLOW 2: SINGLE LINE
122 %
123 =====
124
125 function run_single_line_analysis()
126     % --- Define Physical Constants ---
127     rho_copper = 0.0172;    sigma_copper = 56;
128     rho_aluminum = 0.0282; sigma_aluminum = 36;
129
130

```

```

117 % --- User Input ---
118 fprintf('\n--- Input for Single Line Analysis ---\n');
119 lineTypeChoice = centeredMenu('Select Line Type:', 'Single-Phase', '
Three-Phase');
120 if lineTypeChoice == 1
121     lineType = 'single-phase';
122     U_source = input('Enter single-phase source voltage (L-N) [V] (e
.g., 230): ');
123 else
124     lineType = 'three-phase';
125     U_source = input('Enter three-phase source voltage (L-L) [V] (e.
g., 400): ');
126 end
127 loadCurrent = input('Enter load current [A] (e.g., 25): ');
128 cos_phi = input('Enter load power factor (e.g., 0.9): ');
129 lineLength = input('Enter total line length [m] (e.g., 150): ');
130 reactance = input('Enter line reactance per unit length [Ohm/m] (e.g
., 0.0001): ');
131 frequency = input('Enter AC frequency [Hz] (e.g., 50): ');
132 materialChoice = centeredMenu('Select Conductor Material:', 'Copper
', 'Aluminum');
133 crossSection = input('Enter conductor cross-section [mm^2] (e.g.,
16): ');
134 circuitChoice = centeredMenu('Select Circuit Type (for REBT limits)
:', 'Lighting', 'Power (Other uses)');
135 if materialChoice == 0 || circuitChoice == 0, disp('Invalid
selection. Aborting.');
```

```

136
137 % --- Set Parameters ---
138 switch materialChoice, case 1, resistivity = rho_copper; sigma =
sigma_copper; case 2, resistivity = rho_aluminum; sigma =
sigma_aluminum; end
139 switch circuitChoice, case 1, circuitType = 'lighting'; case 2,
circuitType = 'power'; end
140
141 % --- Resistance Calculations (DC and AC) ---
142 resistance_dc_per_meter = resistivity / crossSection;
143 diameter_m = sqrt(4 * crossSection / pi) / 1000;
144 [k_skin, k_proximity] = A3b_AC_Resistance_Factor(frequency,
diameter_m, sigma);
145 resistance_ac_per_meter = resistance_dc_per_meter * (1 + k_skin +
k_proximity);
146
147 % --- Perform Voltage Drop Calculations ---
148 deltaU_exact_dc = B2_calculateExactVoltageDrop(lineType, lineLength,
loadCurrent, resistance_dc_per_meter, reactance, cos_phi);
149 deltaU_exact_ac = B2_calculateExactVoltageDrop(lineType, lineLength,
loadCurrent, resistance_ac_per_meter, reactance, cos_phi);
150 [isCompliant, percentDrop, limit] = B4_checkREBTCompliance(
deltaU_exact_ac, U_source, circuitType);
151
152 % --- Display Results ---
153 fprintf('\n--- RESULTS for Single Line ---\n');
154 fprintf('DC Resistance per meter: %.5f Ohm/m\n',
resistance_dc_per_meter);
155 fprintf('AC Resistance per meter: %.5f Ohm/m (Skin: %.2f%%, Prox:
%.2f%%)\n', resistance_ac_per_meter, k_skin*100, k_proximity*100);
156 fprintf('-----\n');
```

```

157     fprintf('Voltage Drop (using DC Resistance): %.2f V\n',
        deltaU_exact_dc);
158     fprintf('Voltage Drop (using AC Resistance): %.2f V (%.2f%%)\n',
        deltaU_exact_ac, percentDrop);
159     fprintf('Voltage at Load (based on AC drop): %.2f V\n', U_source -
        deltaU_exact_ac);
160     fprintf('REBT Compliance for %s: %s (Limit: %.1f%%)\n', circuitType,
        string(isCompliant), limit);
161     fprintf('-----\n');
162
163     % --- Plotting ---
164     B5_plotVoltageProfile(U_source, deltaU_exact_dc, deltaU_exact_ac,
        lineLength, circuitType);
165 end

```

D.11 C1_ConductorSizingAnalysis.m

```

1 %
    =====
2 % MAIN SCRIPT for Conductor Cross-Section Sizing (V2 - Integrated)
3 %
    =====
4 % MODIFIED:
5 % - Renamed from C1_CrossSection.m for consistency.
6 % - Integrated into the master project, controlled by '
    sizing_scenarioChoice'.
7 % - The script's workflow changes based on the selected scenario.
8 % - All user menus are now centered.
9 %
    =====
10
11 %% --- Cleanup and Initialization ---
12 % clc; clear; close all; % Controlled by master script
13
14 %% --- Scenario Selection ---
15 if ~exist('sizing_scenarioChoice', 'var')
16     sizing_scenarioChoice = centeredMenu('Select Sizing Criterion:', '
        Voltage Drop (Student A)', 'Current-Carrying Capacity (Student B)', '
        Economic Optimization (Student C)');
17 end
18
19 %% --- Define Physical & Economic Constants ---
20 sigma_copper = 56;
21 sigma_aluminum = 36;
22 LCC_years = 20;
23 hours_per_year = 4000;
24 cost_per_kWh = 0.15;
25
26 %% --- User Input ---
27 disp('--- Conductor Sizing Input ---');
28 lineTypeChoice = centeredMenu('Select Line Type:', 'Single-Phase', '
    Three-Phase');

```

```

29 if lineTypeChoice == 1, lineType = 'single-phase'; else, lineType = '
    three-phase'; end
30 if strcmp(lineType, 'single-phase'), U_source = input('Enter source
    voltage (L-N) [V] (e.g., 230): ');
31 else, U_source = input('Enter source voltage (L-L) [V] (e.g., 400): ');
    end
32 loadCurrent = input('Enter load current [A] (e.g., 45): ');
33 cos_phi = input('Enter load power factor (e.g., 0.9): ');
34 lineLength = input('Enter total line length [m] (e.g., 200): ');
35 vd_limit_percent = input('Enter maximum allowable voltage drop [%] (e.g
    ., 5): ');
36 materialChoice = centeredMenu('Select Conductor Material:', 'Copper', '
    Aluminum');
37 if materialChoice == 1, material = 'Copper'; sigma = sigma_copper; else,
    material = 'Aluminum'; sigma = sigma_aluminum; end
38
39 deltaU_max = U_source * (vd_limit_percent / 100);
40
41 %% --- Execute Analysis Based on Scenario ---
42 switch sizing_scenarioChoice
43     case 1 % Student A: Voltage Drop Criterion
44         fprintf('\n--- ANALYSIS by VOLTAGE DROP Criterion ---\n');
45         s_vd = C2_calculateRequiredSection(lineType, lineLength,
            loadCurrent, cos_phi, sigma, deltaU_max);
46         fprintf('Theoretical section required for voltage drop: %.2f mm
            ^2\n', s_vd);
47         [s_final, ampacity] = C3_selectStandardSection(s_vd, material);
48         fprintf('Selected standard section: %.2f mm^2\n', s_final);
49         if loadCurrent > ampacity
50             fprintf('WARNING: This section may not support the load
                current! (Load: %.1f A, Ampacity: %.1f A)\n', loadCurrent, ampacity);
51         end
52         final_vd = C6_verifyFinalDesign(lineType, lineLength,
            loadCurrent, cos_phi, sigma, s_final, U_source);
53         fprintf('Final calculated voltage drop with %.2f mm^2 section:
            %.2f%%\n', s_final, final_vd);
54
55     case 2 % Student B: Current-Carrying Capacity Criterion
56         fprintf('\n--- ANALYSIS by CURRENT-CARRYING CAPACITY Criterion
            ---\n');
57         [s_final, ~, all_sections] = C3_selectStandardSection(0,
            material); % Get the table
58
59         s_ampacity_options = all_sections([all_sections.ampacity] >=
            loadCurrent, :);
60         if isempty(s_ampacity_options)
61             error('No standard conductor can support the specified load
                current of %.1f A.', loadCurrent);
62         end
63         s_final = s_ampacity_options(1).section; % Select the smallest
            section that meets ampacity
64
65         fprintf('Minimum section required for ampacity (%.1f A): %.2f mm
            ^2\n', loadCurrent, s_final);
66         final_vd = C6_verifyFinalDesign(lineType, lineLength,
            loadCurrent, cos_phi, sigma, s_final, U_source);
67         fprintf('Final calculated voltage drop with this section: %.2f
            %%\n', final_vd);

```

```

68         if final_vd > vd_limit_percent
69             fprintf('WARNING: This section does not meet the voltage
drop requirement! (Calculated: %.2f%%, Limit: %.1f%%)\n', final_vd,
vd_limit_percent);
70         end
71
72         case 3 % Student C: Economic Optimization
73             fprintf('\n--- ANALYSIS by ECONOMIC OPTIMIZATION Criterion ---\n
');
74             % 1. Technical Sizing First
75             s_vd = C2_calculateRequiredSection(lineType, lineLength,
loadCurrent, cos_phi, sigma, deltaU_max);
76             [s_vd_std, ~] = C3_selectStandardSection(s_vd, material);
77
78             [~, ~, all_sections] = C3_selectStandardSection(0, material);
79             s_ampacity_options = all_sections([all_sections.ampacity] >=
loadCurrent, :);
80             if isempty(s_ampacity_options), error('Load current %.1f A is
too high for available conductors.', loadCurrent); end
81             s_amp_std = s_ampacity_options(1).section;
82
83             s_technical = max(s_vd_std, s_amp_std);
84             fprintf('Technically minimum required section: %.2f mm^2\n',
s_technical);
85
86             % 2. Economic Analysis
87             idx_start = find([all_sections.section] == s_technical);
88             sections_to_analyze = all_sections(idx_start:end);
89
90             num_sections = numel(sections_to_analyze);
91             total_costs = zeros(1, num_sections); cable_costs = zeros(1,
num_sections); loss_costs = zeros(1, num_sections);
92
93             for i = 1:num_sections
94                 [total_costs(i), cable_costs(i), loss_costs(i)] =
C4_calculateLifecycleCost(...
95                     sections_to_analyze(i), lineType, lineLength,
loadCurrent, cos_phi, sigma,...
96                     LCC_years, hours_per_year, cost_per_kWh);
97             end
98
99             [min_cost, optimal_idx] = min(total_costs);
100             s_optimal = sections_to_analyze(optimal_idx).section;
101
102             fprintf('-----\n');
103             fprintf('ECONOMIC OPTIMIZATION RESULTS:\n');
104             fprintf('Economically Optimal Section: %.2f mm^2\n', s_optimal);
105             fprintf('Associated Lifecycle Cost: %.2f\n', min_cost);
106             if s_optimal == s_technical
107                 disp('The smallest technically compliant section is also the
most economical.');
```

```

113     C5_plotEconomicAnalysis(sections_to_analyze, cable_costs,
        loss_costs, total_costs, s_technical, s_optimal);
114 end
115
116 if sizing_scenarioChoice == 0, disp('No scenario selected. Exiting.');
```

D.12 centeredMenu.m

```

1 function choice = centeredMenu(title, varargin)
2 %
3 % =====
4 % FUNCTION: centeredMenu (V2 - Robust UI Figure)
5 % =====
6 % Description:
7 % Creates a custom, modal dialog box that is always centered on the
8 % screen.
9 % This version uses uifigure for robust control over position and
10 % behavior,
11 % avoiding issues with the legacy 'menu' function.
12 %
13 % Inputs:
14 % title - The string for the title of the menu box.
15 % varargin - A variable number of strings for the menu options.
16 %
17 % Output:
18 % choice - The index of the selected option (1, 2, 3...), or 0 if
19 % the user closes the window.
20 %
21 % =====
22
23 % --- Define Figure and Button Geometry ---
24 numButtons = length(varargin);
25 figHeight = 50 + numButtons * 40 + 20; % Top padding + buttons + bottom
26 % padding
27 figWidth = 350;
28 buttonHeight = 30;
29 buttonWidth = figWidth - 60; % With padding on each side
30 buttonX = (figWidth - buttonWidth) / 2;
31
32 % --- Create a blank, invisible figure ---
33 fig = uifigure('Visible', 'off', ...
34     'Name', title, ...
35     'WindowStyle', 'modal', ... % Blocks interaction with
36     other windows
37     'Resize', 'off', ...
38     'NumberTitle', 'off', ...
39     'MenuBar', 'none', ...
40     'ToolBar', 'none');
41
42 % --- Center the Figure on the Screen ---
43 screenSize = get(0, 'ScreenSize');
```



```

38 screenWidth = screenSize(3);
39 screenHeight = screenSize(4);
40 figX = (screenWidth - figWidth) / 2;
41 figY = (screenHeight - figHeight) / 2;
42 fig.Position = [figX, figY, figWidth, figHeight];
43
44 % --- Create Buttons for Each Option ---
45 for i = 1:numButtons
46     % Position buttons from the top of the figure down
47     buttonY = figHeight - 50 - ((i-1) * 40);
48
49     uibutton(fig, 'Text', varargin{i}, ...
50         'Position', [buttonX, buttonY, buttonWidth, buttonHeight],
51         ...
52         'ButtonPushedFcn', @(btn, event) buttonCallback(i));
53 end
54 % --- Store the choice in the figure's app data (safer than global) ---
55 fig.UserData.choice = 0; % Default to 0 (if window is closed)
56
57 % --- Define the Nested Callback Function ---
58 function buttonCallback(buttonIndex)
59     % When a button is pushed, store its index and resume the script
60     fig.UserData.choice = buttonIndex;
61     uiresume(fig);
62 end
63
64 % --- Make the figure visible and wait for user input ---
65 fig.Visible = 'on';
66 uiwait(fig); % Pauses execution until uiresume is called or fig is
67             % closed
68 % --- Retrieve the choice and delete the figure to clean up ---
69 if isvalid(fig)
70     choice = fig.UserData.choice;
71     delete(fig);
72 else
73     % This handles the case where the user closes the window with the 'X'
74     choice = 0;
75 end
76
77 end

```